Transformers Models

Models which use Transformers

- There are multiple models which use transformers architecture like BERT, GPT, T5
- But while using this architecture some of these models dont use the entire but only the part of it.
- Like BERT => encoder only
- GPT => decoder only

Why only parts of Transformers ?

- **Computational Complexity:** Using both encoder and decoder increases the model's computational load and memory usage significantly. For example, each layer in the Transformer has its own self-attention and feed-forward layers, so having both encoder and decoder layers means a lot more computation.
- **Balancing Bidirectional and Autoregressive Mechanisms:** In models like BERT (encoder-only) and GPT (decoder-only), the attention mechanisms are tailored to either bidirectional understanding (BERT) or left-to-right generation (GPT). Combining both in a single model requires carefully balancing these mechanisms, which can complicate training and fine-tuning.

Why only parts of Transformers ?

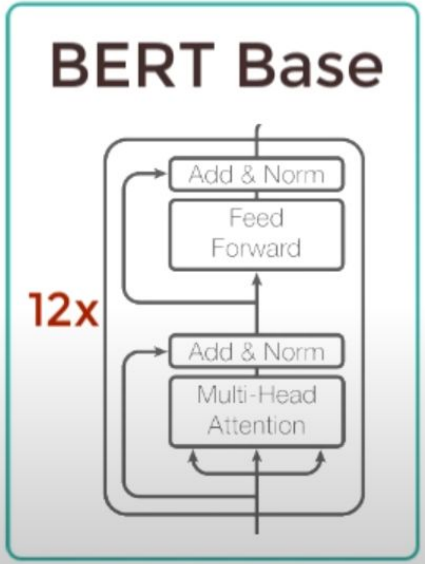
- **Efficiency and Simplicity:** By focusing on either encoding (understanding) or decoding (generating), these models can specialize in their respective tasks without extra computation. For example, BERT doesn’t need to generate text, so including a decoder would be unnecessary overhead. Similarly, GPT doesn’t need to understand entire context bidirectionally, so it skips the encoder.
- **Task Specialization:** BERT, with its encoder-only approach, is excellent for tasks that involve comprehending text. GPT, with its decoder-only approach, is highly effective for generative tasks. They each use only the part of the Transformer that is optimized for their specific tasks.

When: Whole or part of Transformers ?

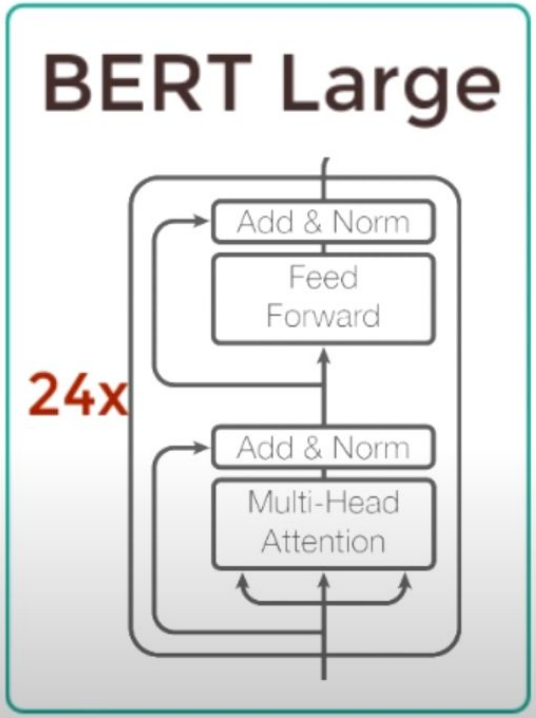
- **Single-Purpose Tasks:** For simpler tasks that involve only understanding (like sentiment analysis) or only generation (like text completion), encoder-only or decoder-only models are more efficient and still perform very well.
- **Sequence-to-Sequence Tasks:** For more complex tasks that need both comprehension and generation (like summarization or translation), models that use both encoder and decoder are more appropriate.

BERT (Bidirectional Encoder Representation from Transformers)

Performance



110M Parameters



340M Parameters

	$BERT_{BASE}$	$BERT_{LARGE}$	Transformer
Layers	12	24	6
Feedforward networks (hidden units)	768	1024	512
Attention heads	12	16	8

BERT

- **BERT** uses only encoders from the transformers
- **Can solve ->**
  Neural machine translation
  Question Answering
  Sentiment Analysis
  Text Summarization
- All these problems needs an understanding of language
- So how to solve these ?
  2 phases
  Pretrain bert to understand language
  Finetune bert to learn specific task

BERT: Overview

- Trained in 2 variations (base - 12 enc - 110m para, large - 24 enc - 340m para)
- Able to handle long input context
- Trained on entire book corpus (800m words) and wiki (2500m words)
- Trained for 1 million steps
- Targeted at multi task objective
- Trained on TPU
- Works for both sentence level and token level task
- Can be finetuned

BERT: Pre-training

- The goal in this is to make bert understand what is language and what is context.
- Bert learns language by training on 2 unsupervised tasks simultaneously
- **Mass language modeling (MLM)**
  In this bert randomly masks words in a sentence
  Like fill in the blanks
  The goal here is to understand the bidirectional context
  Here we mask out k % (recommended 15 %) of the input words and the model is trained to predict these words.
  Too much masking -> not enough context
  Too little masking -> expensive to train

BERT: Pre-training

- **Next sentence prediction (NSP)**
  Here bert takes in 2 sentences and determines if the 2nd sentence actually follows the first
  It helps it understand the context across different sentences
  This is binary classification task.
  Learn the relationship between sentences and predict the next sentence given the first one.

GPT (Generative Pre-trained Transformer)

Decoder-Only

- Decoder-only transformer architectures have taken the spotlight in popular large language models such as GPT-3, GPT-4, PaLM, LaMDa and Falcon.
- The power of Decoder-Only Models lies in their ability to not just mimic human-like text, but to also be creative in their responses. They can craft stories, answer questions, and even engage in dialogue that feels natural and fluid. This capability makes them incredibly useful in a wide range of applications, from chatbots and digital assistants to content creation, abstractive summarization, and storytelling.

GPT

- **1. Foundational Overview of GPT**
  Goal: The core purpose of GPT (Generative Pre-trained Transformer) is to generate human-like text by predicting the next word in a sentence based on a given context. This is achieved by training on a vast amount of text data to capture language patterns.
  Key Insight: Unlike BERT, which is trained bidirectionally (using the context from both sides of a word), GPT is unidirectional, trained to predict the next word from left to right. This unidirectional training makes it ideal for language generation tasks.

GPT Architecture Overview

- GPT is based on the Transformer architecture, but it specifically uses only the decoder stack of the Transformer. This design choice aligns with its purpose of generating text by predicting the next word based on prior context.
- The decoder stack in GPT comprises multiple layers of self-attention and feedforward networks.

GPT Architecture Overview

- **Self-Attention Layer:** Each token in the input sequence can focus on (or "attend to") previous tokens, weighted by their importance. This allows GPT to capture relationships and dependencies across words.
- **Masked Self-Attention:** GPT uses masked self-attention to ensure that tokens only look at preceding tokens when generating text. Masking prevents the model from seeing future tokens, which is crucial for autoregressive generation (i.e., predicting one word at a time in sequence).
- **Feedforward Neural Network:** After self-attention, each token representation is passed through a feedforward network. This network processes the information and refines it, enabling the model to build a more complex understanding of the sequence.
- Each layer of the model repeats this combination of self-attention and feedforward processing multiple times, allowing GPT to build a hierarchical understanding of the input.

Pre-training

- The goal of pre-training in GPT is to develop a broad understanding of language patterns, structures, and semantic relationships.
- **Data Collection:** GPT is trained on a massive, diverse corpus of text from books, websites, and other sources, covering a wide range of topics and styles. This variety enables the model to learn generalized language skills.
- **Objective:** Language Modeling (Next-Token Prediction): The model is trained in an autoregressive manner. This means it's optimized to predict the next word in a sentence given the previous words, a task called language modeling. During training, GPT is exposed to numerous sentence fragments and learns to maximize the probability of the correct next word.
- **Training Strategy:** Using stochastic gradient descent and backpropagation, the model adjusts its internal parameters to minimize prediction errors. Over time, this training allows it to generate increasingly coherent and contextually appropriate text.